Replicating Manifold Steering: Geometry of LLM Representations and Behavior

Kirill Acharya · Stanford University · CS 221M Final Project

Code, checkpoints, and figures: https://github.com/kirillacharya/manifold-steering-replication

Background: two different yet similar LLM spaces and how they are connected

Large language models build up internal representations layer by layer: at layer L, each token maps to a hidden state $h^{(L)} \in \mathbb{R}^d$, and structured concept domains may organize into geometric patterns. Hidden states for *days of the week* or *months of the year* may trace a smooth, low-dimensional manifold $\mathcal{M}_h \subset \mathbb{R}^d$: closed loops for cyclic domains, open curves for sequential ones. Wurgaft et al. [1] ask whether this geometry *predicts behavior*: output probability vectors, embedded via the Hellinger map $p \mapsto \sqrt{p}$ (which makes the Hellinger distance $d_H(p,q) = \frac{1}{\sqrt{2}}\|\sqrt{p} - \sqrt{q}\|$ Euclidean [2]), may trace a matching *behavior manifold* $\mathcal{M}_y$. If $\mathcal{M}_h \approx \mathcal{M}_y$ isometrically (I omit the claims for this in the current work), steering *along* $\mathcal{M}_h$ should produce natural "smooth" behavioral transitions, whereas linear steering cuts off-manifold and "teleports" probability mass. The two strategies interpolate between a source concept $\hat{h}_{c_s}$ and a target $\hat{h}_{c_e}$:

$$\pi_{\text{lin}}(\alpha) = (1-\alpha)\,\hat{h}_{c_s} + \alpha\,\hat{h}_{c_e}, \qquad \pi_{\text{mfd}}(\alpha) = s\big((1-\alpha)\,u_{c_s} + \alpha\,u_{c_e}\big), \quad u_{c_i} = s^{-1}(\hat{h}_{c_i}),$$

where $s : \mathbb{R}^k \to \mathbb{R}^d$ maps intrinsic coordinates back to activation space. We replicate the paper's central figure on a small open model, then extend it to a 2D concept structure (a day×hour torus).

The experiment, step by step

Setup. For all experiments we use instruction-tuned **Gemma-2-2B** ($d = 2304$), patching layer 24 of 26. Two cyclic concept domains: *weekdays* (7 classes) and *months* (12 classes), probed with arithmetic prompts such as "`Q: What day is two days after Monday? A:`" (answer: *Wednesday*), with 6 prompts per concept (`scripts/run_weekdays.py`, `run_months.py`). For each prompt we record the last-token hidden state $h^{(24)}$ and the output distribution over concept

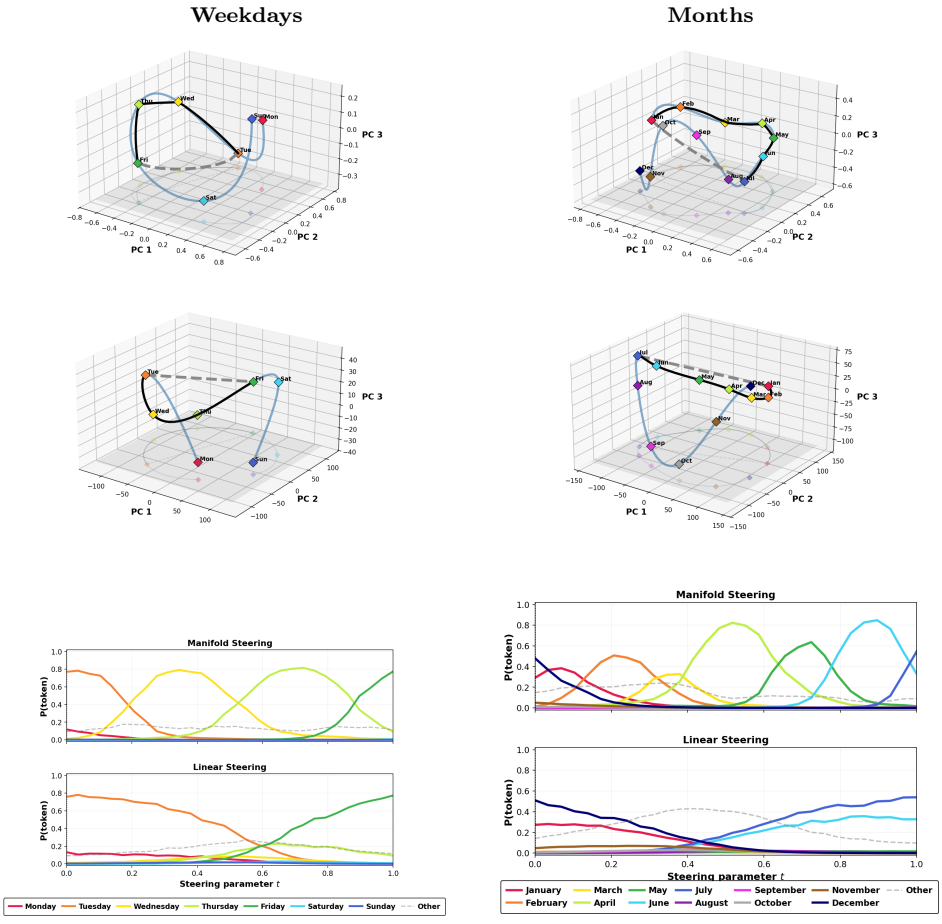

Figure 1: Gemma-2-2B layer 24, steering Tue→Fri (weekdays) and Jan→Jul (months). Rows 1–2: fitted behavior manifolds $\mathcal{M}_y$ (Hellinger–PCA) and activation manifolds $\mathcal{M}_h$ (activation–PCA) with the linear path (dashed) and manifold path (black). Row 3: concept-token probabilities along each path.

tokens, then average per concept: $\hat{h}_c = \frac{1}{N_c}\sum_{i\in S_c} h_i^{(L)}$ and $\hat{p}_c = \frac{1}{N_c}\sum_{i\in S_c} p_i$.

Manifolds. A "manifold" here is simply a smooth curve through the concept centroids. The 7 weekday centroids $\hat{h}_{c_k} \in \mathbb{R}^{2304}$ span at most a 6-dimensional subspace, so PCA to r=6 (r=11 for 12 months) is lossless; it simply re-expresses each centroid in coordinates $z_k \in \mathbb{R}^r$ adapted to the data. We then fit a cubic spline $\gamma : \mathbb{R} \to \mathbb{R}^r$ through the points *in concept order*, with the concept index as curve parameter: $\gamma(k) = z_k$ exactly at each knot, and between knots $\gamma(t) = a_k + b_k(t{-}k) + c_k(t{-}k)^2 + d_k(t{-}k)^3$, coefficients solved jointly so the curve is C^2-smooth. This gives the activation manifold $\mathcal{M}_h$: a single scalar coordinate ("where between Monday and Sunday") decodes to a full hidden state via $\mathrm{PCA}^{-1}(\gamma(t))$. The behavior manifold $\mathcal{M}_y$ is built identically from the probability centroids, except each $\hat{p}_{c_k}$ is first mapped to $\sqrt{\hat{p}_{c_k}}$ (Hellinger space, explained above), where Euclidean tools (PCA, splines, distances) respect the geometry of the probability simplex.

Steering. To steer from a source concept to a target (say Tuesday → Friday, spline indices i_s=1, i_e=4), we build the two paths of the Background section concretely. The *linear* path ignores the manifold: $\pi_{\mathrm{lin}}(\alpha) = (1-\alpha)\hat{h}_{c_s} + \alpha\hat{h}_{c_e}$, a straight segment between the two raw centroids in $\mathbb{R}^{2304}$. The *manifold* path instead interpolates the scalar spline coordinate and decodes: $\pi_{\mathrm{mfd}}(\alpha) = \mathrm{PCA}^{-1}\big(\gamma(i_s + \alpha\,(i_e{-}i_s))\big)$. At $\alpha = \frac{1}{3}$ it passes through a point near the Wednesday centroid, and at $\alpha = \frac{2}{3}$ near Thursday. Both paths are sampled at 30 evenly spaced points α_t. The intervention works like activation patching. We give the model one fixed question (a neutral one, whose correct answer sits midway between source and target, so neither endpoint is favored) and let it process the text normally up to layer 24. At that layer we simply overwrite the hidden state of the final token with our path point $\pi(\alpha_t)$, as if the model itself had computed it. The remaining two decoder blocks and the unembedding then continue from this edited state, and the next-token distribution they produce tells us what the model now "believes": we read off the probability of each concept token (Monday through Sunday) and group everything else into "other" (`scripts/run_steering_probs.py`). Repeating this for all 30 path points traces how behavior changes as the activation slides along each path. If behavior follows geometry, the manifold path should hand probability mass smoothly from concept to concept while the linear path should not. Everything is deterministic; all figures regenerate from committed checkpoints without a GPU (see the repo README).

Replication of the main figure

Figure 1 replicates the paper's main result. In both domains the concept centroids trace closed, ordered loops in activation space and in behavior space; the cyclic structure of weeks and months is visibly encoded. The manifold path stays on the fitted curve and, in the probability trajectories (row 3), produces *smooth, ordered* transitions: steering from Tuesday to Friday passes probability mass through Wednesday and Thursday in sequence. Linear steering instead leaves the manifold and *teleports*: intermediate concepts never rise in order, and off-concept mass appears mid-path. One practical lesson: the patch must be applied deep enough. At layer 12, keys and values computed from the original prompt override the patch over the 14 remaining blocks and both methods flatline, while layer 24 (92% depth) replicates cleanly.

Extension: torus steering (day × hour)

Do LLMs encode *two-dimensional* concept structure? A weekday lives on one circle; a (day, hour) pair lives on a *product* of two circles, which is a torus. We collect layer-24 activations for all 168 prompts "`It is HH:00 on {Day}`" (7 days × 24 hours) and assign each prompt two angles, $\theta_{\mathrm{day}} = 2\pi k/7$ and $\theta_{\mathrm{hour}} = 2\pi h/24$. Each circle's 2D subspace $V_{\mathrm{day}}, V_{\mathrm{hour}} \in \mathbb{R}^{2\times d}$ is recovered by regressing centered activations onto the standardized labels $[\sin\theta, \cos\theta]$ (each label component yields one direction $d \propto \sum_i (h_i - \bar{h})\,\ell_i$), followed by QR-orthogonalization of the two directions (`scripts/run_hierarchical_steering.py`). The two recovered planes are nearly orthogonal to each other (max $|\cos| \approx 0.18$), so day and hour occupy largely independent planes of the residual stream. Projecting each activation onto both planes gives two recovered angles, rendered on a torus surface with day on the major circle and hour on the minor (Fig. 2, left). The 7 day-arcs are cleanly ordered; the hour circle does not fully close, so hour is only partially circular here.

Steering in *product coordinates* respects this factorization: starting from the source activation h_{base} (Tue, 09:00), each step moves the day and hour coordinates *independently within their own planes*, $h(\alpha) = h_{\mathrm{base}} + \Delta_{\mathrm{day}}(\alpha)\,V_{\mathrm{day}} + \Delta_{\mathrm{hour}}(\alpha)\,V_{\mathrm{hour}}$, where $\Delta_c(\alpha)$ interpolates the source→target offset within each plane, the 2D analogue of walking along the manifold. Each patched activation is decoded into a day×hour probability grid: day from token probabilities, hour by nearest centroid in the V_{hour} plane (two-digit hours are multi-token for Gemma, so subspace readout is more faithful). Steering $(\mathrm{Tue}, 09{:}00) \to (\mathrm{Fri}, 15{:}00)$ over six steps (Fig. 2, right), the bright cell walks diagonally across the grid, with both coordinates moving smoothly, while linear steering moves the day correctly but teleports the hour, jumping to 15:00 only at the final step: the same off-manifold failure mode, now visible per coordinate.

Takeaway. Representation geometry is not decoration: moving along it produces natural behavior, and ignoring it breaks transitions exactly as the manifold picture predicts. Code, checkpoints, and full reproduction instructions:

https://github.com/kirillacharya/manifold-steering-replication.

References

[1] D. Wurgaft, C. Rager, M. Kowal, V. Shyam, S. Feucht, U. Bhalla, T. Haklay, E. Bigelow, R. Sarfati, T. McGrath, O. Lewis, J. Merullo, N. D. Goodman, T. Fel, A. Geiger, and E. S. Lubana. Manifold steering reveals the shared geometry of neural network representation and behavior. *arXiv preprint arXiv:2605.05115*, 2026.

[2] S. Amari and H. Nagaoka. *Methods of Information Geometry*. American Mathematical Society, 2000.

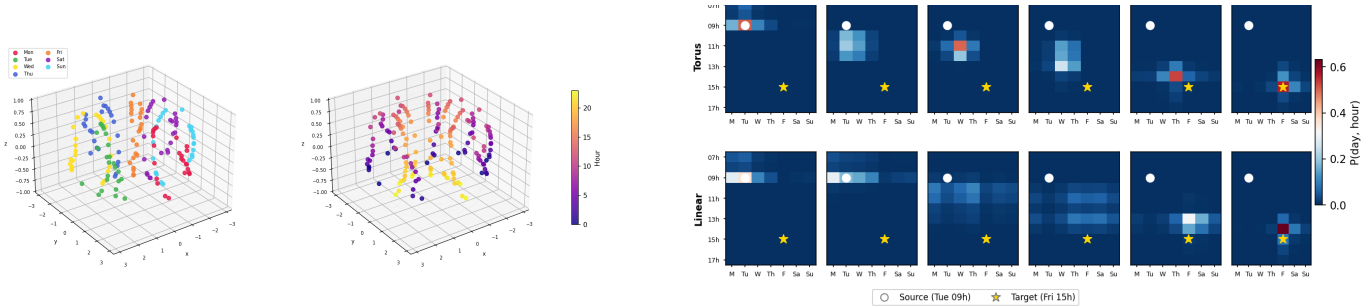

Figure 2: Left: layer-24 activations on the day×hour torus, colored by day (7 ordered arcs) and by hour. **Right:** day×hour probability grids across 6 steering steps, (Tue, 09:00) → (Fri, 15:00); top row torus (product-coordinate) steering, bottom row linear steering.